

Reto 1. Editar tareas y reutilización de código

Implementación de un botón para editar elementos de la lista de compra

Paso 1: Crear el componente del formulario de edición

Paso 2: Integrar la funcionalidad de edición en `ShoppingList`

Paso 3: Manejar la edición en el componente `ShoppingApp`

Refactorización del componente `ItemForm` para manejar creación y edición

Implementación del componente unificado `ItemForm`

Uso del componente `ItemForm`

Implementación de un botón para editar elementos de la lista de compra

Ahora, vamos a enriquecer nuestra aplicación de lista de compra en React añadiendo la funcionalidad de editar elementos. Para ello, implementaremos un botón "Editar" en cada elemento de la lista, que al ser pulsado, mostrará un formulario para editar el elemento seleccionado. Este formulario será un nuevo componente que recibirá el elemento a editar como propiedad.

Paso 1: Crear el componente del formulario de edición

Primero, crearemos un nuevo componente `EditItemForm`, que será responsable de mostrar el formulario de edición para un elemento específico de la lista de compra. Este componente recibirá el elemento a editar como prop y gestionará el proceso de edición.

```
import { useState } from 'react';

function EditItemForm({ item, onSave, onCancel }) {
  const [editedItem, setEditedItem] = useState(item);

  function handleInputChange(event) {
    const { name, value } = event.target;
    setEditedItem({ ...editedItem, [name]: value });
  }

  function handleSubmit(event) {
    event.preventDefault();
    onSave(editedItem);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="text"
        name="name"
        value={editedItem.name}
        onChange={handleInputChange}
        placeholder="Nombre del elemento"
      />
      <input
        type="number"
        name="quantity"
        value={editedItem.quantity}
        onChange={handleInputChange}
        placeholder="Cantidad"
      />
      <button type="submit">Guardar cambios</button>
      <button type="button" onClick={onCancel}>Cancelar</button>
    </form>
  );
}
```

```

    </form>
  );
}

export default EditItemForm;

```

Paso 2: Integrar la funcionalidad de edición en `ShoppingList`

En el componente `ShoppingList`, añadiremos un botón "Editar" a cada elemento de la lista de compra. Al pulsar este botón, se mostrará el formulario `EditItemForm` debajo del elemento seleccionado.

```

import { useState } from 'react';
import EditItemForm from './EditItemForm';

function ShoppingList({ items, onDelete, onEditSave }) {
  const [editingItemIndex, setEditingItemIndex] = useState(null);

  function handleEditClick(index) {
    setEditingItemIndex(index);
  }

  function handleSave(editedItem) {
    onEditSave(editedItem, editingItemIndex);
    setEditingItemIndex(null);
  }

  function handleCancel() {
    setEditingItemIndex(null);
  }

  return (
    <div>
      {items.map((item, index) => (
        <div key={index}>
          <h3>{item.name}</h3>
          <p>Cantidad: {item.quantity}</p>
          <button onClick={() => handleEditClick(index)}>Editar</button>
          <button onClick={() => onDelete(index)}>Eliminar</button>
          {editingItemIndex === index && (
            <EditItemForm
              item={item}
              onSave={handleSave}
              onCancel={handleCancel}
            />
          )}
        </div>
      ))}
    </div>
  );
}

export default ShoppingList;

```

En `ShoppingList`, mantenemos un estado `editingItemIndex` para rastrear qué elemento se está editando. El botón "Editar" establece este índice al índice del elemento correspondiente, y el formulario de edición se muestra solo para el elemento que está siendo editado. Las funciones `handleSave` y `handleCancel` manejan el guardado de cambios y la cancelación de la edición, respectivamente.

Paso 3: Manejar la edición en el componente `ShoppingApp`

Finalmente, en el componente `ShoppingApp`, debemos gestionar el guardado del elemento editado.

```
function ShoppingApp() {
  // ... Estado y funciones previas

  function handleEditSave(editedItem, index) {
    setItems(prevItems => {
      const updatedItems = [...prevItems];
      updatedItems[index] = editedItem;
      return updatedItems;
    });
  }

  // ... Renderizado del componente
}
```

En `handleEditSave`, actualizamos el elemento en la posición correspondiente con los nuevos datos forzando así un nuevo render.

Con estos pasos, hemos añadido con éxito la funcionalidad para editar elementos en nuestra aplicación React de lista de compra. Los usuarios pueden ahora editar elementos existentes de manera eficiente y cómoda, mejorando significativamente la usabilidad de la aplicación.

Refactorización del componente `ItemForm` para manejar creación y edición

Para mejorar la eficiencia y reutilización del código en nuestra aplicación de lista de compra React, vamos a fusionar los componentes `EditItemForm` y `CreateItemForm` en uno solo. Este componente unificado, `ItemForm`, se encargará tanto de añadir nuevos elementos como de editar los existentes. La clave de esta refactorización es inicializar el estado del componente con el elemento a editar, si se proporciona.

Implementación del componente unificado `ItemForm`

El componente `ItemForm` se modificará para aceptar un elemento existente (`itemToEdit`) como prop, que usaremos para inicializar el estado del formulario.

```
import {useState} from 'react';

export default function ItemForm({
  item = {
    product: '',
    quantity: '',
    purchaseDate: '',
    id: `_${Math.random()}`
  },
  onSubmit,
  title
}) {
  const [newItem, setNewItem] = useState(item);

  function handleChange(event) {
    setNewItem((n) => ({
      ...n,
      [event.target.name]: event.target.value
    }));
  }

  function handleSubmit(event) {
```

```

    event.preventDefault();
    onSubmit(newItem);
    if (item.product === '') {
      setNewItem({
        product: '',
        quantity: '',
        purchaseDate: '',
        id: `${Math.random()}`
      });
    }
  }
}

return (
  <div>
    <h2>{title}</h2>
    <div>
      <form onSubmit={handleSubmit}>
        <p>
          <label htmlFor="product">Producto:</label>
        </p>
        <p>
          <input id="product" name="product" value={newItem.product} onChange={handleChange} />
        </p>
        <p>
          <label htmlFor="quantity">Cantidad:</label>
        </p>
        <p>
          <input id="quantity" name="quantity" value={newItem.quantity} onChange={handleChange} />
        </p>
        <p>
          <label htmlFor="purchaseDate">Fecha de compra:</label>
        </p>
        <p>
          <input
            id="purchaseDate"
            name="purchaseDate"
            value={newItem.purchaseDate}
            onChange={handleChange}
          />
        </p>
        <div>
          <button type="submit">Guardar</button>
        </div>
      </form>
    </div>
  </div>
);
}

```

En este código, `ItemForm` se inicializa con `item`. Si `item` no se proporciona, se inicializa con un objeto vacío cuyo `id` lo estamos generando ya de forma aleatoria. Esto nos permite mantener la lógica presente en `CreateItemForm` en donde generábamos el `id` del elemento dentro de la función `handleSubmit`:

```

function handleSubmit(event) {
  event.preventDefault();
  onSubmit({
    ...newItem,
    id: `${Math.random()}`
  });
}

```

```

    });
    setNewItem({
      product: '',
      quantity: '',
      purchaseDate: '',
      id: ''
    });
  }
}

```

Además, hemos añadido una nueva propiedad para poder personalizar el valor del título del formulario y, en el caso de que no estemos editando, el formulario quedará de nuevo vacío tras un envío exitoso:

```

if (item.product === '') {
  setNewItem({
    product: '',
    quantity: '',
    purchaseDate: '',
    id: `${Math.random()}`
  });
}

```

Gracias a esto, hemos agrupado en un sólo componente la lógica de creación y de edición, mejorando tanto la mantenibilidad como la reusabilidad de nuestro código.

Uso del componente `ItemForm`

En el componente `ShoppingList`, usaremos `ItemForm` para agregar nuevos elementos:

```

function ShoppingList() {
  // ... Estado y funciones previas
  return (
    <div>
      <div>
        <ItemForm onSubmit={handleSubmit} title="Crear elemento" />
      </div>
      { /* Resto del componente */ }
    </div>
  );
}

```

En `ShoppingListItems`, usaremos `ItemForm` para editar elementos:

```

return (
  <div>
    <h2>Elementos</h2>
    <div>
      {items.map((item, index) => (
        <div key={item.id}>
          <h4>{item.product}</h4>
          <p>Fecha de compra: {item.purchaseDate}</p>
          <p>Cantidad: {item.quantity}</p>
          <div>
            <button onClick={() => onEditItem(index)}>Editar</button>
            <button onClick={() => onDeleteItem(index)}>Eliminar</button>
          </div>
          {editingItemIndex === index && (
            <div>
              <ItemForm item={item} onSubmit={handleEditItem} title="Editar elemento" />
            </div>
          )}
        </div>
      ))}
    </div>
  </div>
)

```

```
        </div>
      )}
    </div>
  )}
</div>
</div>
);
```

Con esta refactorización, el componente `ItemForm` se convierte en una solución más eficiente y reutilizable, ya que maneja tanto la adición de nuevos elementos como la edición de los existentes. Este enfoque reduce la duplicidad de código y mejora la mantenibilidad, permitiendo una gestión más coherente y efectiva de los formularios en nuestra aplicación React.